Esperimentazioni di Elettronica

Relazione di Laboratorio
A.A. 2024/2025

Giacomo Moretti
Emma Pellizzoni
Leonardo Zambusi

Indice

1 Introduzione e obiettivi del corso **3**

2 Nozioni generali sul microcontrollore **3**
2.1 Manipolazione dei registri . . . 4
2.2 Interrupt . . . 4

3 Periferiche della board **5**
3.1 GPIO digitale . . . 5
3.1.1 Output . . . 5
3.1.2 Input . . . 5
3.2 Timer di base (TIM6) . . . 6
3.3 USART . . . 6
3.3.1 Inizializzazione . . . 6
3.3.2 Trasmissione . . . 7
3.3.3 Ricezione . . . 8
3.4 ADC . . . 10
3.4.1 Inizializzazione . . . 10
3.4.2 Inizio acquisizione e memorizzazione delle misure . . . 11
3.4.3 Conversione di quantità fisiche (termometro e misure di tensione) . . . 11
3.5 DMA . . . 12
3.5.1 Inizializzazione e utilizzo . . . 12
3.5.2 Trasmissione USART . . . 13
3.5.3 ADC . . . 13
3.6 Comparatore analogico . . . 13

4 Progetti introduttivi **14**
4.1 LED e push button . . . 14
4.2 Trasmissione e ricezione con USART . . . 15
4.3 Esponenziali . . . 16
4.4 Monitoraggio temperatura e tensione di riferimento con ADC . . . 18
4.5 Acquisizione di un segnale con DMA, pretrigger e trigger software . . . 20
4.6 Acquisizione di un segnale con DMA, pretrigger e trigger analogico . . . 22

5 Amplificatori **23**
5.1 OP27 . . . 23
5.2 AD848 . . . 25

6 Progetto finale **27**
6.1 Conteggi di buio . . . 27
6.2 Misurazioni finali . . . 28

7 Approfondimenti **31**
7.1 Coincidenze . . . 31
7.2 Cristalli . . . 31

Riferimenti bibliografici **34**

1 Introduzione e obiettivi del corso

L'obiettivo del corso è l'introduzione alle principali componenti di una catena di misurazione con rivelatori di particelle, ovverosia amplificatori, convertitori da analogico a digitale, microcontrollori con gestione di trigger e trasmissioni dati, rivelatori SiPM e l'utilizzo di strumentazione di supporto, quale l'oscilloscopio. In particolare il corso si suddivide in una parte in cui la finalità è quella di comprendere il funzionamento delle principali periferiche del microcontrollore per la gestione gestione dei dati e la struttura del microcontrollore stesso e in un'altra in cui si costruisce una vera propria catena di misurazione di fotoni emessi da un led lampeggiante, con il fine di costruire lo spettro discreto di emissione. Inoltre nella relazione si riporta anche la descrizione e l'analisi dati compiuta in approfondimento riguardante la ricezione di fotoni emessi da un cristallo scintillatore in presenza di una sorgente radioattiva con trigger e la coincidenza di ricezione di due fotomoltiplicatori.

La relazione di laboratorio è strutturata nel seguente modo: si compie una panoramica generica sul microcontrollore utilizzato, illustrandone la struttura, come si opera su di esso, in particolare sui registri e come si compiono le differenti opzioni riguardanti le operazioni bitwise. Nella seconda sezione si descrivono invece in modo generico le periferiche utilizzate nel microcontrollore, illustrando porzioni di codice che sono caratteristici del funzionamento di quest'ultime. Queste porzioni di codice verranno utilizzate ripetutamente nelle esperienze introduttive o nel progetto finale, ma non verranno ripetuti e rispiegati in seguito. Infine si riportano le esperienze introduttive al progetto finale e questo stesso con i relativi approfondimenti. In queste sezioni non vengono riportati interamente i codici scritti, ma la logica di implementazione del codice tramite una schematica ad alto livello attraverso pseudo codice. Non vengono inseriti i codici per scorrevolezza nella lettura, ma vengono poste nel testo delle porzioni che sono ritenute essere significative per l'esperienza.

Nella relazione vengono illustrate delle metodologie risolutive dei problemi proposti nel corso; tuttavia ne esistono di diverse. Per questa motivazione non è detto che le strutture algoritmiche coincidano in toto con i codici implementati dagli studenti Moretti, Pellizzoni e Zambusi durante la durata del corso. I codici completi che sono stati implementati durante il corso vengono riportati in GitHub. Si riporta dunque una risoluzione generica al problema, che nel caso specifico dello studente viene modificata o implementata in modo diverso, senza però cambiare sostanzialmente di significato.

Vengono invece riportate le analisi dati compiute per la caratterizzazione dei principali elementi circuitali eseguita prima di utilizzare la configurazione finale per l'acquisizione dello spettro in Drive.

2 Nozioni generali sul microcontrollore

Il microcontrollore utilizzato durante l'intero corso di laboratorio è STM32H743, un ARM Cortex-M7, su una scheda applicativa Nucleo-144 [1, pp. 44, 45]. È già predisposta una periferica di debug [1, pp. 28, 29], ST-LINK v3, che permette di collegare il microcontrollore a un computer tramite USB, caricare il programma e avviare una sessione di debug. Per quanto riguarda le nozioni teoriche principali riguardanti il microcontrollore (periferiche, pin) si rimanda al manuale utente della scheda [2] e alle slides [3, 1].
Il linguaggio di programmazione utilizzato è il C e come ambiente di sviluppo CubeIDE [4]. Questo software permettere di compilare, caricare il programma e compiere un debug direttamente dalla stessa finestra; inoltre mette a disposizione il configuratore CubeMX con il quale è possibile generare il codice di inizializzazione del microcontrollore, gestione del clock e attivazione e configurazione dei pin e delle periferiche.

Per interagire con il microcontrollore e le sue periferiche è possibile utilizzare la libreria CMSIS (*Common Microcontroller Software Interface Standard*), l'HAL (*Hardware Abstraction Layer*) oppure scrivere codice a basso livello accedendo direttamente ai registri. La libreria CMSIS offre un'interfaccia comune a tutti i microcontrollori ARM Cortex-M, includendo funzioni come `__enable_irq()` e `__disable_irq()` che consentono di abilitare o disabilitare globalmente

gli interrupt. L'HAL, invece, facilita l'interazione con le periferiche dell'STM32H743, mettendo a disposizione diverse funzionalità come la trasmissione e la ricezione di dati. Il codice di inizializzazione generato da CubeMX si basa sull'HAL.
Per scopi didattici, nonché per una maggiore flessibilità e prestazioni, il codice sviluppato è scritto a basso livello, interagendo direttamente con l'hardware.

2.1 Manipolazione dei registri

Di seguito vengono illustrate le operazioni principali che si svolgono per manipolare i registri. In particolare viene illustrato qui di seguito come svolgere le principali operazioni bitwise, quali fare il set e il clear di un bit senza modificare gli altri e scrivere un valore in un bitfield di un registro senza modificare gli altri.

```
// Scrittura di un registro
MY->REGISTER = 4; // Scrivo 4 in REGISTER

// Manipolazione di un signolo bit
#define BIT5_POS  5
#define BIT5_MASK  (1 << BIT5_POS) // Seleziona il bit 5
MY->REGISTER |= MASK_BIT5; // Scrivo 1 nel bit 5, lasciando invariati gli altri
MY->REGISTER &= ~MASK_BIT5; // Scrivo 0 nel bit 5

// Scrittura di un bitfield a più bit
#define BITFIELD_POS 2
#define BITFIELD_MASK (7 << BITFIELD_POS) // Seleziona i bit 2, 3 e 4
MY->REGISTER &= ~BITFIELD_MASK; // Cancello il contenuto dei bit 2, 3 e 4
MY->REGISTER |= value << BITFIELD_POS; // Scrivo 'value' nei bit 2, 3 e 4

// Lettura di un registro
uint32_t val = MY->REGISTER;

// Lettura di un bitfield
uint32_t bitfield = (MY->REGISTER & BITFIELD_MASK) >> BITFIELD_POS;

// Controllare se un bit è impostato o vuoto
if (MY->REGISTER & MASK_BIT5)
    ; // bit impostato
else if (~(MY->REGISTER) & MASK_BIT5)
    ; // bit non impostato
```

In alcuni casi è previsto che per cancellare un bit si debba scrivergli 1 anziché 0. In questo caso è opportuno non usare un'operazione *bitwise or*: si andrebbe a scrivere 1 in tutti i bit impostati nel registro, anziché solo nella posizione del bit che si desidera cancellare. È quindi opportuno implementare il codice come segue.

```
// Cancellare il flag di un registro scrivendoci dentro 1
MY->STATUS = STATUS_FLAG; // Scrivo un solo 1 al bit FLAG nel registro STATUS
```

2.2 Interrupt

Gli interrupt permettono di eseguire una porzione di codice per reagire a un evento hardware. Sono funzioni che possono essere chiamate in qualunque momento interrompendo l'esecuzione del codice principale. Al termine dell'esecuzione dell'interrupt, il flusso di controllo viene ripristinato [3, pp. 23–26]. Tipicamente, una funzione di interrupt è condivisa per più eventi e talvolta per più periferiche. È necessario quindi indicare in opportuni registri di configurazione quali eventi comportano l'invocazione dell'interrupt. Durante l'esecuzione dell'interrupt è possibile determinare quali eventi si sono verificati leggendo i relativi bit negli status register. È necessario che tali bit vengano svuotati per segnalare la corretta gestione dell'evento, o dall'hardware o agendo da software scrivendo un registro.

Nei codici di esempio riportati nelle sezioni successive si riportano delle funzioni interrupt per ogni evento. Nel codice completo, queste funzioni vengono chiamate dalle funzioni interrupt definite da CubeIDE controllando il verificarsi dell'evento desiderato.

3 Periferiche della board

In questa sezione vengono spiegate le principali funzionalità delle periferiche utilizzate all'interno del corso. Vengono riportati porzioni di codici caratteristici delle periferiche utilizzate i quali vengono utilizzati ripetutamente all'interno delle esperienze compiute, che vengono illustrate nelle sezioni successive.

3.1 GPIO digitale

La periferica GPIO (*General Purpose Input/Output*) permette di controllare individualmente dei pin hardware sulla scheda. Limitando la discussione ai pin digitali, questi possono essere impostati come input o output. I pin sono organizzati in porte, ognuna delle quali gestisce 16 pin fisici.

Si può utilizzare CubeMX per impostare i pin in modalità input o output. È anche possibile configurare altre impostazioni, ma per le attività che seguono non è stato necessario.

3.1.1 Output

I pin di output permettono di forzare una tensione alta o bassa tramite codice. È possibile leggere e impostare lo stato del pin agendo sul bit relativo del registro ODR (*Output Data Register*). Di seguito il codice che viene utilizzato per compiere tale operazione.

```
// Leggo stato del pin PB0
bool pinState = (GPIOB->ODR & GPIO_ODR_OD0) != 0;
// Forzo il pin PB0 a alto (lettura ODR, OR e scrittura ODR)
GPIOB->ODR |= GPIO_ODR_OD0;
// Cambio stato pin PB0 (lettura ODR, XOR e scrittura ODR)
GPIOB->ODR ^= GPIO_ODR_OD0;
```

Siccome la scrittura del registro ODR prevede di riscrivere lo stato di tutti i pin di quella porta è necessario prima leggere il contenuto di ODR per evitare di alterare involontariamente lo stato di altri pin. Questo rende l'operazione lenta, perché sono necessarie diverse operazioni, suscettibile a errori e non atomica, ovverosia non è eseguita in un solo colpo di clock. Ciò può introdurre problemi di sincronizzazione se si modifica lo stato di pin della stessa porta in interrupt.

Pertanto conviene usare il registro BSRR (*Bit Set/Reset Register*), il quale consente operazioni atomiche.

```
// Forzo il pin PB0 alto (operazione atomica)
GPIOB->BSRR = GPIO_BSRR_BS0;
// Forzo il pin PB0 basso (operazione atomica)
GPIOB->BSRR = GPIO_BSRR_BR0;
// Cambio lo stato del pin PB0 (lettura ODR, decisione e scrittura BSRR)
GPIOB->BSRR = (GPIOB->ODR & GPIO_ODR_OD0) ? GPIO_BSRR_BS0 : GPIO_BSRR_BR0;
```

LED La scheda STM32H7 Nucleo-144 mette a disposizione tre LED utente: il verde collegato a PB0, il giallo a PE1 e il rosso a PB14. I LED sono accesi se il pin digitale è alto e sono normalmente spenti. Normalmente spento significa che c'è una resistenza di pull-down sull'ingresso: se il pin non forza una tensione, allora la tensione in uscita è zero [5, p. 5].

3.1.2 Input

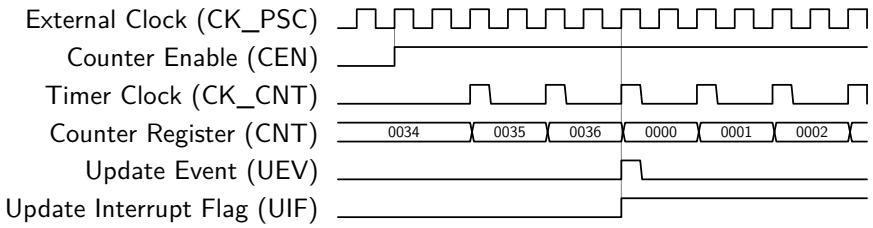
I pin di input digitali permettono di distinguere gli stati alto e basso, ovvero se sono collegati a massa o all'alimentazione. È possibile leggere lo stato del pin dal registro IDR (*Input Data Register*). È invece possibile impostare su CubeMX il pin come EXTI (*Extended Interrupt and Event Controller*), funzionalità che permette di chiamare un interrupt al verificarsi di un evento di rise, fall o entrambi [6, §§ 20.6.1, 20.6.2].

Push button La scheda Nucleo-144 mette a disposizione anche un push button utente collegato al pin PC13. Il pin è alto quando il pulsante è premuto ed è normalmente basso [5, p. 3].

3.2 Timer di base (TIM6)

Il microcontrollore mette a disposizione diversi timer [3, pp. 28, 29], ovverosia un contatore a 16 bit a ricarica automatica con prescaler programmabile [6, § 42]. È utile per generare un evento a intervalli regolari[1].

Il seguente diagramma mostra il principio di funzionamento.[2]



Per ogni colpo di CK_CNT, il registro CNT (*Counter*) viene incrementato. Se CNT raggiunge il valore memorizzato nel registro ARR (*Auto-Reload Register*), al clock successivo viene generato l'evento UEV (*Update Event*) e CNT viene azzerato [6, § 42.3.2]. Il registro PSC (*Prescaler*) regola la frequenza di CK_CNT dividendo quella di CK_PSC di un fattore PSC + 1 [6, §§ 42.3.1, 42.4.7].

Operativamente, per usare il timer TIM6 è necessario:

- abilitarlo su CubeMX, attivando l'interrupt;
- configurare i valori dei registri PSC, ARR ed eventualmente porre CNT = 0;
- abilitare l'interrupt dell'evento update, impostando il flag UIE (*Update Interrupt Enable*) del registro DIER (*DMA/Interrupt Enable Register*);
- abilitare il timer, impostando il flag CEN del registro CR1.

La funzione interrupt viene chiamata alla generazione dell'evento update, quando CNT raggiunge ARR e viene azzerato. All'interno dell' interrupt è necessario notificare l'avvenuta gestione dell'evento cancellando il bit UIF (*Update Interrupt Flag*) del registro SR (*Status Register*) [6, § 42.4.4]. È possibile generare l'evento di update e riportare CNT a zero da software impostando il bit UG del registro EGR (*Event Generation Register*).

È possibile sfruttare l'evento di update come evento di trigger esterno per altre periferiche. Ciò si ottiene impostando l'opzione *Trigger Event Selection* su CubeMX.

3.3 USART

La periferica USART (*Universal Synchronous/Asynchronous Receiver Transmitter*) permette di mettere in comunicazione il microcontrollore con un dispositivo esterno, in questo caso il computer e in particolare il software Matlab [6, § 48][3, pp. 47–71].

3.3.1 Inizializzazione

Nella scheda applicativa, la porta USB di debug è collegata all'USART3 [2, § 6.6.5]. Il computer rileva una porta seriale virtuale con la quale è possibile comunicare.

Sul configuratore CubeMX è necessario abilitare la periferica, impostare la velocità di trasmissione (*baud rate*) e abilitare l'interrupt se desiderato.

Per quanto riguarda il codice si deve abilitare la periferica e le funzionalità di ricezione e trasmissione; si selezionano inoltre quali eventi chiamino l'interrupt. Si riporta di seguito la porzione di codice che compie ciò che viene descritto sopra.

[1] Spegnere e accendere un LED o campionare tramite ADC ad esempio

[2] Diagramma basato sulla figura 518 del manuale [6], con PSC = 1 e ARR = 36.

```c
void usart_setup(void)
{
    // Abilito ricezione e trasmissione
    USART3->CR1 |= USART_CR1_RE | USART_CR1_TE;
    // Abilito interrupt per eventi RXNE e TC
    USART3->CR1 |= USART_CR1_RXNEIE | USART_CR1_TCIE;
    // Abilito la periferica
    USART3->CR1 |= USART_CR1_UE;
}
```

3.3.2 Trasmissione

La trasmissione di un byte si effettua scrivendolo nel registro TDR (*Transmit Data Register*). Quando la trasmissione è terminata, il microcontrollore abilita il flag TC (*Transmission Complete*) nel registro ISR (*Interrupt and Status Register*) ed è possibile inviare il byte successivo [6, § 48.5.5]. Il flag viene cancellato scrivendo nuovamente in TDR o impostando il bit corrispondente nel registro ICR (*Interrupt Flag Clear Register*) [6, § 48.8.10]. La funzionalità può essere facilmente implementata in modo sospensivo. Di seguito un esempio di codice per la trasmissione di caratteri.

```c
void usart_send_sync(void* ptr, size_t dim)
{
    uint8_t* my_ptr = (uint8_t*)ptr;
    for (size_t i = 0; i < dim; i++)
    {
        // Trasmetto un byte
        USART3->TDR = my_ptr[i];

        // Attendo fine trasmissione
        while ((~USART3->ISR) & USART_ISR_TC);
    }

    // Cancello il flag TC da ISR
    USART3->ICR = USART_ICF_TCCF;
}
```

L'utilizzo degli interrupt per la trasmissione tramite USART comporta la necessità di cambiare contesto tra funzioni, richiedendo quindi una maggiore attenzione nella progettazione e gestione dei dati. Tra le possibili strategie per affrontare questa complessità si possono considerare:

- Scrivere i dati da trasmettere in un vettore globale dedicato e avviare la trasmissione;
- Fornire, di volta in volta, l'indirizzo dei dati da trasmettere;
- Implementare un sistema di code (queue) per gestire i dati in modo strutturato.

Viene quindi presentata di seguito l'implementazione della strategia che fornisce l'indirizzo di memoria dei dati da trasmettere, scelta per la sua buona flessibilità d'uso e per la semplicità nella realizzazione.

```c
// Indirizzo dei dati da trasmettere
static uint8_t* volatile usart_tx_ptr = NULL;
// Dimensione in byte dei dati indirizzati da usart_tx_ptr
static volatile size_t usart_tx_dim;
// Posizione del prossimo carattere da trasmettere
static volatile size_t usart_tx_pos;

void usart_send(void* ptr, size_t dim)
{
    assert(usart_tx_ptr == NULL);

    usart_tx_ptr = (uint8_t*)ptr;
    usart_tx_dim = dim;
    usart_tx_pos = 1;

```

```
    // trasmetto il primo byte
    USART3->TDR = usart_tx_ptr[0];
}

void usart_wait_send_completion(void)
{
    while (usart_tx_ptr);
}

void usart_interrupt_tc(void)
{
    if (usart_tx_pos < usart_tx_dim)
    {
        // trasmetto il byte successivo
        USART3->TDR = usart_tx_ptr[usart_tx_pos++];
    }
    else
    {
        // non ho altro da scrivere: cancello il flag TC
        USART3->ICR = USART_ICR_TCCF;
        void* last_ptr = usart_tx_ptr;
        usart_tx_ptr = NULL;

        void usart_send_complete(void*); // Implementato dall'utente
        usart_send_complete(last_ptr);
    }
}
```

L'uso della funzione `usart_send` richiede alcune accortezze da parte dell'utente:

- Assicurarsi che non ci siano altre trasmissioni in esecuzione;
- Assicurarsi che l'indirizzo passato rimanga valido durante tutta la trasmissione (ad esempio allocato come globale o dinamicamente);
- Implementare la funzione `usart_send_complete` che permette di reagire quando la trasmissione è terminata.

3.3.3 Ricezione

Quando l'USART riceve un byte, questo viene scritto nel registro RDR (*Receive Data Register*), attivando l'evento RXNE (*Receive Data Register Not Empty*). Leggere il contenuto di RDR disabilita RXNE [6, § 48.8.10]. Se viene ricevuto un secondo byte mentre RDR è ancora pieno, viene generato un evento di overrun (ORE, *Overrun Error*).

Nel contesto delle esperienze svolte, i dati ricevuti possono essere di due tipi:

- Singoli caratteri di controllo al quale il microcontrollore deve reagire. Un esempio è il comando di avvio acquisizione;
- Informazioni composte da più byte, come valori di configurazione[3].

Come nel caso della trasmissione, il secondo punto richiede di definire una strategia per gestire i dati ricevuti e renderli accessibili al chiamante. Una possibilità è quella di usare un buffer globale in cui salvare i byte ricevuti e alla chiamata di una funzione leggere i valori ricevuti.

```
static volatile bool usart_listen_to_commands = true;
static volatile uint8_t usart_rx_buffer[1024];
static volatile size_t usart_rx_pos;

void usart_interrupt_rxne(void)
{
    // legge il byte ricevuto
    uint8_t c = (uint8_t)USART3->RDR;

    if (usart_listen_to_commands)
```

[3] Per esempio, nella sezione 4.3 vengono trasferiti da Matlab alcuni valori di configurazione.

```
    {
        void usart_handle_command(uint8_t); // Implementato dall'utente
        usart_handle_command(c);
    }
    else
    {
        assert(usart_rx_pos < sizeof(usart_rx_buffer));
        usart_rx_buffer[usart_rx_pos++] = c;
    }
}

void usart_read(void* ptr, size_t dim)
{
    assert(!usart_listen_to_commands);

    // aspetta di ricevere tutti i byte
    while (usart_rx_pos < dim);
    // copia i dati ricevuti nel buffer del chiamante
    memcpy(ptr, (uint8_t*)usart_rx_buffer, dim);

    // elimina i caratteri letti da usart_rx_buffer
    __disable_irq(); // disabilita interrupt per evitare che usart_rx_pos cambi
    usart_rx_pos -= dim;
    assert(usart_rx_pos >= 0);
    if (usart_rx_pos > 0)
    {
        memmove(
            (uint8_t*)usart_rx_buffer,
            (uint8_t*)usart_rx_buffer + dim,
            usart_rx_pos);
    }
    __enable_irq(); // riabilita interrupt
}

void usart_read_disable_commands(void)
{
    usart_rx_pos = 0;
    usart_listen_to_commands = false;
}

void usart_read_enable_commands(void)
{
    usart_listen_to_commands = true;
}
```

L'implementazione proposta risulta sufficientemente flessibile, ma richiede di dimensionare il buffer globale in base alla dimensione massima dei pacchetti che si intende ricevere con una singola chiamata a `usart_read` e gestire correttamente l'abilitazione dei comandi. Di seguito viene riportato un esempio che illustra come usare le funzioni introdotte.

```
enum { STATE_IDLE, STATE_READ_CONFIG } gState = STATE_IDLE;

// Funzione chiamata nell'interrupt RXNE
void usart_handle_command(uint8_t c)
{
    switch (c)
    {
    case 'r':
        // Evento che si può gestire velocemente
        GPIOB->BSRR = GPIO_BSRR_BS14; // accendo il LED rosso
        return;
    case 's':
        // Evento che richiede computazioni nel main
        gState = STATE_READ_CONFIG;
        usart_read_disable_commands(); // dovrò usare usart_read
        return;
    default:
    }
}
```

```
20
21 int main(void)
22 {
23     /* ... */
24
25     uint16_t my_config1 = 0, my_config2 = 0;
26     while (1)
27     {
28         switch (gState)
29         {
30             case STATE_READ_CONFIG:
31                 usart_read(&my_config1, sizeof(my_config1));
32                 usart_read(&my_config2, sizeof(my_config2));
33
34                 usart_read_enable_commands();
35                 gState = STATE_IDLE;
36                 break;
37             /* ... */
38         }
39     }
40 }
```

3.4 ADC

La periferica ADC (*Analog to Digital Converter*) permette di convertire un segnale analogico in un segnale digitale [7, p. 27]. Gli ADC messi a disposizione dal microcontrollore sono a 16 bit ad approssimazioni successive [6, § 25] [7, pp. 41, 42, 43]. Ogni ADC è collegato a diversi canali, a loro volta collegati a pin GPIO analogici oppure a canali interni dedicati, ad esempio per il monitoraggio della temperatura [6, §§ 25.4.4, 25.4.33].

Il modello di utilizzo prevede di preselezionare i canali che si intendono misurare, specificando anche il tempo per il campionamento. Successivamente si accende e calibra l'ADC. A questo punto si definisce la sequenza di misure da eseguire. Ricevuto un evento di inizio sequenza, l'ADC esegue una misura e una conversione per ogni canale. Il microcontrollore memorizza il risultato nel registro DR (*Data Register*) e attiva il flag EOC (*End Of Conversion*). Prima che l'ADC memorizzi la prossima conversione, è necessario leggere quella corrente o scartarla, altrimenti si verifica la condizione di overrun, segnalata dal flag OVR. È possibile gestire automaticamente questo evento abilitando OVRMOD (*Overrun Mode*) per sovrascrivere il valore precedente oppure AUTDLY (*Delayed Conversion Mode*) per attendere la lettura del valore precedente prima di cominciare una nuova conversione. L'evento di trigger per cominciare un'acquisizione può essere hardware, ad esempio generato all'evento UE di un timer, oppure software [8, pp. 60, 61]. Nel caso sia abilitato CONT (*Continuos Mode*), l'ADC comincia autonomamente una nuova acquisizione al termine della precedente fino a quando non viene fermato manualmente. [6, § 25.4]

3.4.1 Inizializzazione

Su CubeMX si abilitano gli ingressi dell'ADC collegati ai canali che si intendono usare.[4] Dal pannello *GPIO Settings* è possibile determinare quale pin del microcontrollore corrisponde. Sempre da CubeMX si può configurare inoltre la sorgente di trigger esterno[5] e, attraverso la finestra *Clock Configuration*, il clock dell'ADC. Infine è possibile abilitare la funzione interrupt.

Prima di usare l'ADC è necessario accenderlo e calibrarlo. Successivamente si preselezionano i canali, si imposta il sample rate e si definisce la sequenza. Si riporta un esempio di codice che illustra i diversi passaggi.

```
1 void adc_enable(void)
2 {
3     // Svuoto status register
```

[4] Nelle esperienze si sono sempre usati canali single-ended, in quanto si era interessati a campionare dei segnali rispetto al pin di massa.

[5] Nelle esperienze nelle sezioni 4.5 e 4.6 viene usato l'evento di UEV del TIM6 come sorgente di trigger esterno per le acquisizioni ADC.

```
    ADC3->ISR = 0xFFFFFFFF;
    // calibrazione per misura non differenziale (single-ended)
    ADC3->CR &= ~ADC_CR_ADCALDIF;
    // calibrazione lineare
    ADC3->CR |= ADC_CR_ADCALLIN;
    // disabilita ADC
    ADC3->CR &= ~ADC_CR_ADEN;
    // avvio calibrazione
    ADC3->CR |= ADC_CR_ADCAL;

    // attendi fine calibrazione
    while (ADC3->CR & ADC_CR_ADCAL);

    // azzera flag scrivendogli 1
    ADC3->ISR = ADC_ISR_ADRDY;
    // accendi ADC
    ADC3->CR |= ADC_CR_ADEN;

    // attendi che si accenda
    while (~ADC3->ISR & ADC_ISR_ADRDY);
    ADC3->ISR = ADC_ISR_ADRDY;

    // abilita interrupt di fine conversione
    ADC3->IER |= ADC_IER_EOCIE;
}

int main(void)
{
    // [...]

    adc_enable();

    // Preseleziona canale 13
    ADC3->PCSEL |= ADC_PCSEL_PCSEL_13;

    // Imposta sample time del canale 13 a 810.5 colpi di clock
    ADC3->SMPR2 &= ~ADC_SMPR2_SMP13_Msk;
    ADC3->SMPR2 |= 7 << ADC_SMPR2_SMP13_Pos;

    // Imposta sequenza { 13 }
    ADC3->SQR1 = 0;
    ADC3->SQR1 |= 13 << ADC_SQR1_SQ1_Pos;
    ADC3->SQR1 |= 0 << ADC_SQR1_L_Pos;

    // [...]
}
```

3.4.2 Inizio acquisizione e memorizzazione delle misure

Per cominciare ad acquisire delle misure è necessario impostare il bit ADSTART del registro CR. Nel caso in cui il trigger sia software, le conversioni partono immediatamente e alla fine della sequenza ADSTART viene cancellato. Nel caso di trigger hardware, le conversioni partono al verificarsi dell'evento impostato e ADSTART rimane abilitato anche dopo la fine della sequenza. È possibile interrompere un'acquisizione in corso impostando il bit ADSTOP.

Il dato campionato è disponibile alla fine di ogni conversione, segnalata dal bit EOC dello status register. Nel caso si sia abilitato l'interrupt per l'evento EOC e la sequenza sia composta da più conversioni, è necessario gestire autonomamente una macchina a stati per tenere traccia di quale sia lo slot a cui appartiene il dato memorizzato in DR.[6]

3.4.3 Conversione di quantità fisiche (termometro e misure di tensione)

Il microcontrollore mette a disposizione un sensore per monitorare la sua temperatura interna; quello genera una tensione linearmente dipendente alla temperatura misurata. L'uscita VSENSE

[6] In questi casi risulta vantaggioso abilitare anche l'interrupt EOS, che segnala la fine della sequenza. In questo modo si evitano problemi in caso una sequenza venga interrotta prematuramente.

è collegata all'ADC3. È possibile convertire il valore letto T_{adc} nella temperatura T_{celsius}[8, pp. 20, 21]:

$$T_{\text{celsius}} = \frac{110\,^\circ\text{C} - 30\,^\circ\text{C}}{T_2 - T_1}(T_{\text{adc}} - T_1) + 30\,^\circ\text{C},$$

dove T_1 e T_2 sono i valori ritornati dall'ADC alle temperature di 30 °C e 110 °C rispettivamente, misurati in fase di calibrazione in fabbrica[8, p. 22].

Per usare VSENSE è necessario abilitare il termometro, ovverosia scrivere il bit TSEN nel registro CCR dell'ADC3, e abilitare il canale di ingresso dell'ADC con CubeMX.

I dati letti dall'ADC (V_{adc}) possono essere convertiti in tensione usando la formula[8, pp. 25–28]:

$$V_{\text{volt}} = \frac{3.3\,\text{V}}{2^{16} - 1} V_{\text{adc}}.$$

Il microcontrollore fornisce anche una sorgente di una tensione di riferimento (VREFINT) che si può usare per calibrare il fondo scala delle misure. La configurazione su CubeMX è analoga a quella di VSENSE. Secondo il manuale il microcontrollore richiede un tempo di acquisizione elevato per effettuare la misura ($t_{\text{S_vrefint}} > 4.3\,\mu\text{s}$, [9, § 7.3.5]). Effettuata la misura della tensione di riferimento con l'ADC $V_{\text{ref adc}}$, una misura di un canale analogico V_{adc} può essere convertita nel suo valore di tensione V_{volt}:

$$V_{\text{volt}} = \frac{V_{\text{ref volt}}}{V_{\text{ref adc}}} V_{\text{adc}},$$

dove $V_{\text{ref volt}} = (1.22 \pm 0.04)\,\text{V}$ è il valore a manuale della tensione di riferimento generata [9, § 7.3.5].

3.5 DMA

Il DMA (*Direct Memory Access*) permette di trasferire dati direttamente tra periferica e memoria senza passare per il processore. Il microcontrollore mette a disposizione due controller indipendenti con otto stream ciascuno [8, pp. 42–51].

3.5.1 Inizializzazione e utilizzo

Da CubeMX si abilita, per la periferica desiderata, uno stream DMA. Il configuratore preimposta autonomamente la direzione dello stream (da periferica a memoria o viceversa) e le dimensioni dei dati. È necessario inoltre impostare la periferica per usare il DMA.[7] Nel codice si alloca un vettore e si configura il controller DMA fornendo l'indirizzo del vettore e la dimensione. La modalità di funzionamento predefinita è detta lineare, ovvero il vettore in RAM viene percorso una volta e, raggiunta la fine, si disabilita. È possibile abilitare la modalità circolare che permette al DMA di ripartire dall'inizio del vettore una volta raggiunta la fine. Con il seguente esempio di codice si mostra come usare il DMA [8, pp. 55–59, 62, 63].

```
uint16_t buffer[100];
void dma_init_example(void)
{
    // Imposto indirizzo registro periferica
    DMA1_Stream0->PAR = &ADC3->DR;

    // Imposto indirizzo e dimensione del vettore allocato per DMA
    DMA1_Stream0->M0AR = buffer;
    DMA1_Stream0->NDTR = sizeof(buffer) / sizeof(buffer[0]);

    // Cancello tutti i flag di stato legati allo stream
    DMA1->LIFCR = DMA_LIFCR_CFEIF0 | DMA_LIFCR_CDMEIF0
        | DMA_LIFCR_CTEIF0 | DMA_LIFCR_CHTIF0 | DMA_LIFCR_CTCIF0;

    // Abilito DMA circolare (commentare per DMA lineare)
    DMA1_Stream0->CR |= DMA_SxCR_CIRC;
```

[7] Nella sezione 3.5.2 e sezione 3.5.3 vengono trattate nello specifico USART e ADC.

```
17
18      // Abilito il DMA
19      DMA1_Stream0->CR |= DMA_SxCR_EN;
20
21      // [...]
22
23      // Per disabilitare il DMA:
24      DMA1_Stream0->CR &= ~DMA_SxCR_EN;
25  }
```

Il registro NDTR, oltre che essere usato in fase di inizializzazione per impostare la dimensione del vettore, durante la trasmissione viene decrementato ed è possibile ricavare l'indice del prossimo elemento da scrivere: `i = DIM_VETTORE - NDTR`.

È possibile abilitare l'interrupt del DMA per l'evento TC (*Transfer Complete*), chiamato quando si raggiunge la fine del vettore e quando viene disabilitato il DMA durante la trasferimento.

3.5.2 Trasmissione USART

È possibile utilizzare il DMA per trasmettere il contenuto di un vettore tramite USART. Si configura il DMA in modalità lineare come descritto nel paragrafo precedente e si imposta il bit DMAT del registro CR3 dell'USART.

Quando viene abilitato il DMA, i dati vengono trasferiti autonomamente, senza bisogno di gestire gli eventi dell'USART manualmente.

3.5.3 ADC

Il DMA impostato sulla periferica ADC permette di automatizzare due possibili casi di utilizzo:

- *Normal mode:* se il DMA è impostato in modalità lineare, l'ADC acquisice un numero fisso di valori fino a riempire il vettore di destinazione.
- *Scan mode:* se il DMA è impostato in modalità circolare, l'ADC può acquisire dati in modo continuo, sovrascrivendo i dati. Al verificarsi di un evento, si hanno a disposizione anche le misure che lo precedono.

È necessario impostare il campo DMNGT del registro CFGR dell'ADC per l'uso del DMA lineare o circolare.

3.6 Comparatore analogico

Il comparatore analogico confronta due tensioni riportando tramite lo status register quale delle due è maggiore [8, pp. 89–98].

La modalità di utilizzo usata nelle esperienze prevede di confrontare un segnale esterno con una tensione di riferimento, generata con un DAC (*Digital to Analog Converter*). Si usa CubeMX per abilitare il DAC e il comparatore; si configura uno dei due input del comparatore con l'uscita del DAC. Si può impostare un livello di isteresi. Sempre da CubeMX si ottiene il nome del pin a cui è associato l'ingresso analogico.

```
1  void comp_init(void)
2  {
3      // Abilita comparatore
4      COMP1->CFGR |= COMP_CFGRx_EN;
5
6      // Abilita DAC
7      DAC1->CR |= DAC_CR_EN1;
8      // Imposta tensione 1 V
9      DAC1->DHR12R1 = (uint16_t)(1.0/3.3 * (1 << 12));
10     // Applica nuova tensione
11     DAC1->SWTRIGR |= DAC_SWTRIGR_SWTRIGR1;
12 }
```

Lo stato del comparatore è ottenibile dal flag CxVAL del registro SR.

4 Progetti introduttivi

In questa sezione vengono illustrati tutti i progetti introduttivi, compiuti tramite microcontrollori, con finalità didattica alla comprensione del funzionamento del dispositivo. Per ogni esperienza si riportano gli obiettivi e la logica di implementazione del codice, tramite una schematica ad alto livello con pseudo codice; non si riporta il codice completo per scorrevolezza di lettura.

Per quanto riguarda i dettagli riguardanti il funzionamento e l'implementazione codice riguardante le periferiche si rimanda a ciò che è stato scritto nella sezione precedente (sezione 3). Vengono riportati in GitHub le soluzioni implementate durante il corso dagli studenti; come già precedentemente specificato in alcuni casi le modalità di risoluzione utilizzate non ricalcano perfettamente quanto illustrato nelle seguenti sezioni, ma contengono delle differenze seppur non cambiando sostanzialmente il senso dell'esperienza.

4.1 LED e push button

L'esperienza consiste nell'accensione di due LED in modo intermittente a una frequenza prestabilita, che viene cambiata alla pressione di un pulsante. Uno degli obiettivi consiste nell'esecuzione di più operazioni in simultanea, quali il lampeggiamento del LED e la ricezione dell'input della pressione del pulsante. In secondo luogo si vuole prendere confidenza con il microcontrollore. Nel corso dell'esperienza sono state provate diverse strategie, tra cui il delay, il polling e la gestione degli interrupt. Il delay consiste nel bloccare l'esecuzione per un tempo predeterminato, consumando colpi di clock con un numero opportuno di istruzioni vuote o con l'ausilio di un timer hardware. Questa tecnica è detta sospensiva, ovvero blocca l'esecuzione di altre operazioni. Il polling prevede il controllo periodico esplicito riguardo l'avvenimento di un evento. È possibile eseguire più operazioni in simultanea, ma la frequenza dei controlli è determinata dalla velocità di esecuzione degli altri compiti. L'interrupt permette al microcontrollore di eseguire una porzione di codice al verificarsi di un evento: il controllo è eseguito via hardware e l'esecuzione del programma principale viene interrotta in qualunque momento consentendo, a costo di un piccolo overhead per il cambio di contesto, di gestire l'evento immediatamente.

Viene presentata la versione finale del programma, che si basa su interrupt. I pin dei LED verde e rosso sono stati abilitati in modalità output, mentre il pin del push-button è stato configurato in modalità EXTI. È stato utilizzato il TIM6 per generare gli eventi regolari, che gestiscono l'accensione e lo spegnimento dei LED.

Algoritmo 1: Implementazione LED e pulsante con interrupt

```
1  Function main():
2      CubeMX generated initialization
3      Configure TIM6
4          Set initial ticking frequency
5          Enable UIE interrupt
6          Start counter
7      Loop
           // Infinite loop
8  Interrupt TIM6():                                   // Called at regular intervals
9      State Machine ledState
10         State 0
11             Turn green LED on and red LED off
12             Next state
13         State 1
14             Turn green LED off and red LED on
15             Reset to state 0
16     Clear interrupt flag
17 Interrupt EXTI13():                          // Called when push button is pressed
18     State Machine speedState
19         State 0, 1, 2, 3                  // Each state corresponds to different speeds
20             Change TIM6 speed
21             Next state
22         State 4
23             Change TIM6 speed
24             Reset to state 0
25     Generate TIM6 update event        // This prevents CNT to be bigger than ARR
26     Clear interrupt flag
```

4.2 Trasmissione e ricezione con USART

L'esperienza consiste nella trasmissione e nella ricezione di dati tra microcontrollore e Matlab tramite l'USART, con la verifica della corretta ricezione tramite l'utilizzo di un Echo e della velocità di trasmissione come benchmark delle prestazioni del sistema. È stata abilitata nel configuratore la periferica ed è stato impostato il baud rate di 921 600 bit/s. Il benchmark trasmette 256 kB in 3.361 s (76.2 kB/s).

Algoritmo 2: USART: echo

```
1  Function main():
2      CubeMX generated initialization
3      Configure USART3
4          Enable RX and TX
5          Enable RXNE and TC interrupts
6      Loop
7          usart_read_disable_commands()
8          character ← usart_read()
9          usart_send(character)
10         usart_wait_send_completion()
```

Algoritmo 3: USART: velocità trasmissione

```
1  Function main():
2      CubeMX generated initialization
3      Configure USART3
4          Enable TX
5          Enable TC interrupts
6      Configure TIM6
7          PSC ← 32000
8          ARR ← 2^16 − 1
9          Enable UIE interrupt
10     array ← fill with large arbitrary data to transmit
       // Measure time needed to transmit array
11     Start TIM6
12     usart_send(array)
13     usart_wait_send_completion()
14     Stop TIM6
15     cnt ← TIM6 counter
16     usart_send(cnt)
17     Loop
           // Infinite loop
```

4.3 Esponenziali

L'esperienza consiste nella generazione di numeri che seguono un andamento nato dalla sottrazione di due esponenziali $e^{-x/\tau}$ con due costanti τ differenti e nell'invio dei numeri generati a Matlab tramite USART. Gli obiettivi dell'esperienza consistono nella simulazione numerica sul microcontrollore e nell'individuazione di un metodo di comunicazione efficiente tra microcontrollore e Matlab per lo scambio di dati e istruzioni.

L'algoritmo 4 mostra il principio di funzionamento del codice sul microcontrollore. Nell'implementazione in C è stata fatta una scelta in merito alla rappresentazione dei dati sui quali si eseguono operazioni aritmetiche frequenti, nell'esempio le moltiplicazioni delle righe 25 e 26. Nell'ipotesi in cui il microcontrollore non abbia una FPU (*Floating Point Unit*) [3, pp. 39–45], le operazioni su float risultano significativamente più dispendiose rispetto a somma, moltiplicazione e bitshift su interi.[8] Si è quindi scelto di rappresentare `exp1`, `exp2`, `exp1_dt` e `exp2_dt` come interi in virgola fissa, dove i primi 6 bit sono la parte intera e gli ultimi 10 quella decimale. Nel codice 1 viene mostrata l'implementazione di queste ottimizzazioni.

[8] Nel STM32H743 è presente una FPU, quindi eseguire operazioni su float o su interi potrebbe non introdurre differenze di prestazioni significative, se non addirittura risultare più vantaggioso usare i float: a seconda di come è implementata la FPU, è possibile che una moltiplicazione tra float sia eseguita in un colpo di clock, mentre la versione con gli interi comporta oltre alla moltiplicazione delle operazioni di bitshift e cast. Per scopo didattico, si è ipotizzato che non fosse disponibile una FPU.

Algoritmo 4: Esponenziali: codice microcontrollore

```
Function main():
    CubeMX generated initialization
    Configure USART3
        Enable RX and TX
        Enable RXNE and TC interrupts
    gState ← IDLE
    Loop
        State Machine gState
            State IDLE
                // Do nothing
            State CONFIG
                n_points ← usart_read()
                τ1 ← usart_read()
                τ2 ← usart_read()
                exp1 ← usart_read()
                exp2 ← usart_read()
                n_points ← usart_read()
                i ← 0
                // Precalculate exp(dt) factors, using
                exp1_dt ← exp(−1/τ1)
                exp2_dt ← exp(−1/τ2)
                usart_read_enable_commands()
                gState ← IDLE
            State RUN
                exp_data[i] ← exp1 - exp2                    // int16_t
                exp1 ← exp1 × exp1_dt
                exp2 ← exp1 × exp2_dt
                i ← i + 1
                if i ≥ n_points then
                    usart_send(exp_data)
                    usart_wait_send_completion()
                    usart_read_enable_commands()
                    gState ← IDLE
Function usart_handle_command(command):        // Called by usart_interrupt_rxne
    switch command do
        case 'c' do
            gState ← CONFIG
        case 'r' do
            gState ← RUN
        otherwise do
            return
    usart_read_disable_commands()
```

Di seguito un esempio di implementazione di codice lato Matlab.

```
// Line 14
float exp1_float;
usart_read(&exp1_float, sizeof(exp1_float));
exp1 = (uint16_t)(exp1_float * (1 << 10));

```

```
// Line 18
exp1_dt = (uint16_t)(exp(-1.0 / tau1) * (1 << 10));

// Line 25
exp1 = (uint16_t)(((uint32_t)exp1 * (uint32_t)exp1_dt) >> 10);
```

Codice 1: Esponenziali: implementazione in C di ottimizzazioni di alcune righe dell'algoritmo 4.

```
n = 100;
tau1 = 15;
tau2 = 10;
init1 = 63.477;
init2 = 63.477;

mycomm = serialport('/dev/tty.usbmodem103', 921600);

write(mycomm, 'c', 'char'); % send config command
write(mycomm, n, 'uint32');
write(mycomm, tau1, 'single');
write(mycomm, tau2, 'single');
write(mycomm, init1, 'uint16');
write(mycomm, init2, 'uint16');

write(mycomm, 'r', 'char'); % send run command
vals = read(mycomm, n, 'int16');

delete(mycomm)

y = vals / 2^10;
plot(y);
grid on;
```

Codice 2: Esponenziali: codice Matlab.

4.4 Monitoraggio temperatura e tensione di riferimento con ADC

La seguente esperienza consiste nella misurazione della temperatura interna del microcontrollore e della tensione di riferimento tramite ADC. I dati raccolti sono poi inviati a Matlab tramite USART. Lo scopo dell'attività è quello di prendere dimestichezza con l'ADC, gestire sequenze di acquisizione con più conversioni e convertire i valori grezzi nelle rispettive quantità fisiche.[9]

I dati inviati tramite seriale possono essere grezzi oppure già convertiti nelle quantità finali o in un formato intermedio. Eseguire un passaggio di conversione sul microcontrollore permette di inviare dati che non richiedono informazioni aggiuntive per essere interpretati, come valori di calibrazione e impostazioni dell'ADC, e quindi introduce un livello di astrazione che rende indipendente il codice ricevente dall'hardware che ha effettuato la misura. Inoltre, un formato intermedio permette di comprimere l'informazione inviata, ad esempio riducendo il campo di misura o la risoluzione. Si ha però un potenziale costo in prestazioni che può essere significativo o meno a seconda della complessità dei calcoli e di quante volte vengono eseguiti; questo è da confrontare con eventuali requisiti come, ad esempio, garantire un tempo minimo tra un'acquisizione e l'altra. La scelta di come rappresentare i dati da inviare è quindi orientata a trovare un bilancio tra manutenibilità complessiva del progetto e prestazioni.

A scopo didattico, in questa esperienza i dati di temperatura sono convertiti sul microcontrollore e spediti come intero con unità di misura $\mathrm{d^\circ C} = 0.1\,^\circ\mathrm{C}$ (codice 3), mentre la tensione di riferimento viene trasmessa come dato grezzo dell'ADC e poi convertita su Matlab usando come fondo scala 3.3 V.

L'algoritmo 5 illustra il codice del microcontrollore. Il codice 4 è il codice Matlab di comunicazione e conversione dei dati ricevuti.

[9] Per la trattazione sulla configurazione dell'ADC, dei sensori e sulle conversioni dei dati acquisiti si rimanda alla sezione 3.4, in particolare 3.4.3.

Algoritmo 5: Monitoraggio con ADC: codice microcontrollore

```
Global variable: gVoltageData (uint16_t[])
Global variable: gTemperatureData (uint16_t[])
Global variable: gNumberOfSlotsRead (uint32_t)
Function main():
    CubeMX generated initialization
    Configure USART3
        Enable RX and TX
        Enable RXNE and TC interrupts
    Configure ADC3
        Calibrate and enable
        Enable EOC interrupt
        Enable continuous mode
        Preselect VREFINT and TSENSE channels and set maximum sample time
        Setup sequence: slot 1 → VREFINT, slot 2 → TSENSE
    gState ← IDLE
    Loop
        State Machine gState
            State IDLE
                // Do nothing
            State MEASURE
                usart_read_disable_commands()
                gNumberOfSlotsRead ← 0
                Start ADC acquisition
                gState ← MEASURING
            State MEASURING
                // Do nothing here:
                // wait for EOC to change state when all data has been read
            State TRANSMIT
                usart_send(gVoltageData)
                convertedTemperatureData ← convert gTemperatureData in 1/10 of °C
                usart_send(convertedTemperatureData)
                usart_wait_send_completion()
                usart_read_enable_commands()
                gState ← IDLE
Function usart_handle_command(command):        // Called by usart_interrupt_rxne
    if command = 'm' then
        gState ← MEASURE
        usart_read_disable_commands()
Interrupt ADC3():                                          // Called on EOC event
    gNumberOfSlotsRead ← gNumberOfSlotsRead + 1
    currentMeasure ← DR register
    if gNumberOfSlotsRead = 1 (mod 2) then
        Append currentMeasure to gVoltageData
    else
        Append currentMeasure to gTemperatureData
    // Stop after taking 10 measurements
    if gNumberOfSlotsRead ≥ 2 · 10 then
        Stop ADC
        gState ← TRANSMIT
    Clear interrupt flag
```

Di seguito la porzione di codice da lato microcontrollore per la conversione dei dati della temperatura.

```
uint16_t convertedTemperatureData[N_MEAS];

for (uint32_t i = 0; i < N_MEAS; i++)
{
    uint16_t cal30 = *TEMPSENSOR_CAL1_ADDR;
    uint16_t cal110 = *TEMPSENSOR_CAL2_ADDR;

    uint32_t conv = (1100-300) * (uint32_t)gTemperatureData[i] / (cal110-cal30)
        + 300;

    convertedTemperatureData[i] = (uint16_t)conv;
}
```

Codice 3: Monitoraggio con ADC: implementazione in C della conversione della temperatura nell'algoritmo 5, riga 24.

Di seguito un esmepio di codice Matlab per la conversione dei dati per la tensione di riferimento.

```
mycomm = serialport('/dev/tty.usbmodem103', 921600);

write(mycomm, 'r', 'char');
voltages_raw = read(mycomm, 10, 'uint16');
temperatures_dC = read(mycomm, 10, 'uint16');

voltages_V = 3.3 / (2^16 - 1) * voltages_raw
temperatures_C = temperatures_dC / 10

delete(mycomm)
```

Codice 4: Monitoraggio con ADC: codice Matlab.

4.5 Acquisizione di un segnale con DMA, pretrigger e trigger software

L'attività prevede l'acquisizione di un segnale analogico con una frequenza di campionamento superiore a 1 MHz. I dati acquisiti tramite l'ADC vengono successivamente trasmessi mediante interfaccia USART. L'inizio del segnale di interesse viene rilevato da un evento di trigger, determinato in questo caso dal superamento di una soglia con isteresi. Inoltre, è richiesto l'invio di alcuni campioni acquisiti prima dell'evento di trigger, noti come *pretrigger*.

Per soddisfare i requisiti di velocità, il clock del microcontrollore è stato configurato al valore massimo supportato (480 MHz). L'avvio della conversione ADC è gestito da un timer (TIM6) impostato alla frequenza di campionamento desiderata. La memorizzazione dei dati acquisiti è affidata a un DMA circolare, che garantisce un'acquisizione continua, la quale consente di conservare anche i dati precedenti all'evento di trigger, oltre a ridurre il carico sulla CPU.

Il sistema di trigger è implementato nell'interrupt EOC (*End Of Conversion*) dell'ADC, confrontando il valore acquisito con le soglie di riferimento. All'interno dello stesso interrupt, viene inoltre monitorato il numero di campioni acquisiti dopo il trigger, al fine di interrompere l'acquisizione e avviare la trasmissione dei dati. Anche l'invio dei dati tramite USART è gestito tramite DMA, ottimizzando così la velocità di trasmissione. L'implementazione del codice del microcontrollore è schematizzata con l'algoritmo 6; la ricezione dati e la rappresentazione tramite Matlab è gestita come mostrato nel codice 5.

Algoritmo 6: Acquisizione di un segnale con trigger software: codice microcontrollore

```
   Global variable: gSignalData (uint16_t[])                    // Last element holds trigger position
   Global variable: gTriggerState (WAIT_HYSTERESIS, ARMED, TRIGGERED)
   Global variable: gPostTriggerCount (uint16_t)
 1 Function main():
 2     CubeMX generated initialization
 3     Configure USART3
 4         Enable RX
 5         Enable RXNE interrupt
 6     Configure TIM6
 7         PSC ← 0
 8         ARR ← 31
 9     Configure ADC3
10         Calibrate and enable
11         Enable continuous mode
           // VSIGNAL: the channel connected to the pin with the external signal
12         Preselect VSIGNAL and set minimum sample time
13         Setup sequence: slot 1 → VSIGNAL
14         Use TIM6 as trigger source
15         Set DMA mode to circular
16     start_adc()
17     Loop
           // Infinite Loop
18 Function start_adc():
19     Configure DMA ADC
20         Clear status register flags
21         M0AR ← address of gSignalData
22         NDTR ← length of gSignalData − 1
23         Enable circular mode
24         Enable stream
25     Configure ADC3
26         Disable EOC interrupt                  // EOC will be enabled when user sends a character
27     Start ADC3
28     Start TIM6
29 Function start_transmission():
30     Configure DMA USART
31         Clear status register flags
32         M0AR ← address of gSignalData
33         NDTR ← length of gSignalData
34         Enable TC interrupt
35         Enable stream
36 Function usart_handle_command(command):                  // Called by usart_interrupt_rxne
37     if command = 'a' then
38         usart_read_disable_commands()
39         Configure ADC3
40             Enable EOC interrupt
41         gTriggerState ← WAIT_HYSTERESIS
42 Interrupt ADC3():                                        // Called on EOC event
43     State Machine gTriggerState
44         State WAIT_HYSTERESIS
45             if DR < LOW THRESHOLD then
46                 gTriggerState ← ARMED
47         State ARMED
48             if DR ≥ HIGH THRESHOLD then
49                 gSignalData[lastElement] ← DMA ADC NDTR
50                 gTriggerState ← TRIGGERED
51                 gPostTriggerCount ← 0
52         State TRIGGERED
53             gPostTriggerCount ← gPostTriggerCount + 1
54             if gPostTriggerCount ≥ POST TRIGGER LENGTH then
55                 Stop TIM6 and ADC3
56                 Disable DMA ADC
57                 start_transmission()
58     Clear interrupt flag
59 Interrupt DMA USART():                                   // Called on TC event
60     Disable DMA USART
61     start_adc()
62     Clear interrupt flag
```

```
SAMPLE_LENGTH = 100; POST_TRIGGER_LENGTH = 75;
PSC = 0; ARR = 31; TIM_CLK = 120e6;
sample_frequency = TIM_CLK / (PSC + 1) / (ARR + 1);
time_step_sec = 1 / sample_frequency;
time_step_usec = time_step_sec * 10^6;

mycomm = serialport('/dev/tty.usbmodem103', 921600);
write(mycomm, 'a', 'char');
raw_sample = read(mycomm, SAMPLE_LENGTH, 'uint16');
pivot = read(mycomm, 1, 'uint16');
delete(mycomm)

idx_pretrig = mod(POST_TRIGGER_LENGTH - pivot - 1, SAMPLE_LENGTH) + 1;
ordered_sample = [ raw_sample(idx_pretrig:end) raw_sample(1:idx_pretrig) ];
voltages_V = 3.3 / (2^16 - 1) * ordered_sample;
time_scale = time_step_usec * (POST_TRIGGER_LENGTH + (-SAMPLE_LENGTH:0));
plot(time_scale, voltages_V)
```

Codice 5: Acquisizione di un segnale: codice Matlab.

4.6 Acquisizione di un segnale con DMA, pretrigger e trigger analogico

L'esperienza consiste nell'acquisizione di un segnale analogico utilizzando un comparatore analogico per rilevare l'evento di trigger. Questo approccio differisce dall'attività precedente, descritta nella sezione 4.5, in cui il trigger era attivato confrontando i valori letti dall'ADC con soglie predefinite. Il principio di funzionamento resta lo stesso di quanto spiegato in precedenza, mentre le differenze riguardano principalmente la configurazione iniziale del comparatore analogico, descritta in sezione 3.6, e la logica del sistema di trigger, illustrata nell'algoritmo 7. Il codice Matlab rimane invariato (codice 5).

Algoritmo 7: Acquisizione di un segnale con trigger analogico: logica di trigger

```
Global variable: gSignalData (uint16_t[])      // Last element holds trigger position
Global variable: gTriggerState (WAIT_HYSTERESIS, ARMED, TRIGGERED)
Global variable: gPostTriggerCount (uint16_t)
Interrupt ADC3():                                          // Called on EOC event
    State Machine gTriggerState
        State WAIT_HYSTERESIS
            if C1VAL flag is not set in COMP SR then
                gTriggerState ← ARMED
        State ARMED
            if C1VAL flag is set in COMP SR then
                gSignalData[lastElement] ← DMA ADC NDTR
                gTriggerState ← TRIGGERED
                gPostTriggerCount ← 0
        State TRIGGERED
            gPostTriggerCount ← gPostTriggerCount + 1
            if gPostTriggerCount ≥ POST TRIGGER LENGTH then
                Stop TIM6 and ADC3
                Disable DMA ADC
                start_transmission()
    Clear interrupt flag
```

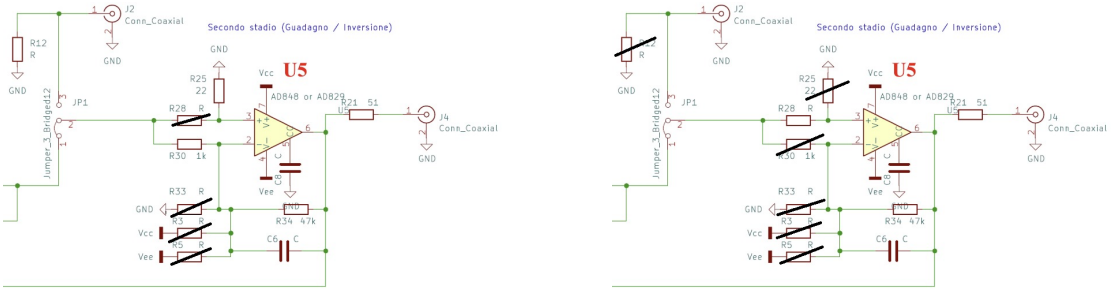


Figura 5.1: Configurazione invertente (sinistra) e non invertente (destra).

5 Amplificatori

L'obiettivo dell'esperienza è la comprensione del funzionamento di un amplificatore operazionale. In particolare, in questa sezione si vanno a caratterizzare due amplificatori, uno a singolo polo dominante, OP27, e uno a doppio polo dominante, AD848 [10, pp. 59, 71]. Si caratterizzano gli amplificatori in configurazione sia invertente, sia non invertente [10, pp. 68, 63]. Si riporta nella figura 5.1 le schematizzazioni dei circuiti utilizzati delle configurazioni invertente e non invertente.

Per caratterizzazione di un amplificatore si intende la misurazione delle principali caratteristiche e del confronto con i valori teorici calcolati a priori; in particolare si misurano la frequenza di taglio e il guadagno. Si utilizzano tre diverse strategie per la misurazione delle due quantità; si generano tramite un oscilloscopio un'onda quadra, un impulso sinusoidale e del rumore bianco. Tramite la generazione delle prime due funzioni d'onda si calcola il guadagno come rapporto di ampiezza uscente ed entrante. Per la misurazione della frequenza di taglio, con la generazione di impulso sinusoidale si calcola a priori l'ampiezza teorica legata alla frequenza di taglio e si ricerca la frequenza di generazione corrispondente; con l'impulso di onda quadra si misura il tempo di salita o di discesa dell'esponenziale in uscita (10 % – 90 %, poi convertito) e si ricava la frequenza di taglio tramite l'inverso; con il rumore bianco si misura il valore di frequenza calcolando l'intersezione tra la trasformata di Fourier del segnale in uscita e la retta corrispondente a -3 dB dal segnale iniziale in uscita, corrispondente al calcolo teorico. Per quanto riguarda i calcoli teorici si rimanda alle slides [10, pp. 39–51].

5.1 OP27

Si riporta qui di seguito le misurazioni e i calcoli compiuti con l'amplificatore OP27. Si riportano solo le misurazioni e i calcoli significativi per l'analisi compiuta ai fini di ottenere una maggior scorrevolezza nella lettura[10]. Le percentuali di confronto riportate sono dovute al rapporto tra la misurazione compiuta e il calcolo teorico, dato che è impossibile effettuare un confronto statistico per mancanza di stima d'errore sulle misurazioni.

Configurazione	**Guadagno**	**Bandwidth** (MHz)
Invertente	10.00	0.73
	4.67	1.41
	1.00	4.00
Non Invertente	2.00	4.00
	5.67	1.41
	7.83	1.02

Tabella 5.1: Risultati teorici ricavati dalle misurazioni delle resistenze

[10] I dati completi dell'OP27 e i calcoli sono consultabili sul foglio di calcolo.

ONDA QUADRA			
Guadagno	**Stima**	**Frequenza di taglio (MHz)**	**Stima**
9.66	96.63%	0.83	113.86%
4.49	96.21%	1.71	121.05%
1.01	100.83%	4.31	107.64%
1.98	99.21%	2.19	54.69%
5.55	97.97%	1.68	119.12%
7.71	98.44%	1.19	116.52%

FFT	
Frequenza taglio (MHz)	**Stima**
1.04	143.51%
1.74	123.02%
3.67	91.82%
3.38	84.43%
1.60	113.12%
1.10	107.86%

IMPULSO SINUSOIDALE			
Guadagno	**Stima**	**Frequenza taglio (MHz)**	**Stima**
10.23	102.34%	1.00	137.50%
4.64	99.52%	1.72	121.83%
0.99	99.17%	3.67	91.75%
2.01	100.30%	3.47	86.75%
5.65	99.62%	1.64	116.17%
7.74	98.83%	1.15	112.60%

Tabella 5.2: Stima delle frequenze di taglio con onda quadra (sopra), FFT (centro) e impulso sinusoidale (sotto).

Si nota che le percentuali di accettabilità riportate indichino un discostamento dai valori predetti teoricamente rispetto alle misure delle frequenze di taglio. Per quanto riguarda la stima della frequenza di taglio con un impulso di onda quadra, l'oscilloscopio misura il tempo di salita dell'esponenziale considerando il massimo; tuttavia, non si ottiene una forma d'onda ben definita e dunque la misurazione non risulta essere precisa. Nella figura 5.2 si mostra un'acquisizione che presenta quanto descritto sopra.

In riferimento alla figura 5.3, si è svolta una simulazione utilizzando l'applicazione LTSpice per verificare l'effettivo comportamento dell'amplificatore OP27, che talvolta non rispecchia l'andamento previsto per un amplificatore a singolo polo ideale. Riprodotte le medesime condizioni, il risultato confuta ogni dubbio mostrandoci la compatibilità tra le misure sperimentali e simulate.

In figura 5.5 si può osservare che l'FFT ottenuta è soggetta a rumore, aggiungendo incertezza nella stima dell'interesezione dello spettro con il guadagno di riferimento.

Infine per le misurazioni della frequenza di taglio con impulso sinusoidale la precisione dell'oscilloscopio nel ricercare l'ampiezza corrispondente non era sufficiente per ottenere valori molto accurati, per questa motivazione in alcuni casi si può osservare una percentuale di accettabilità che indica un discostamento tra valori teorici e misurati.

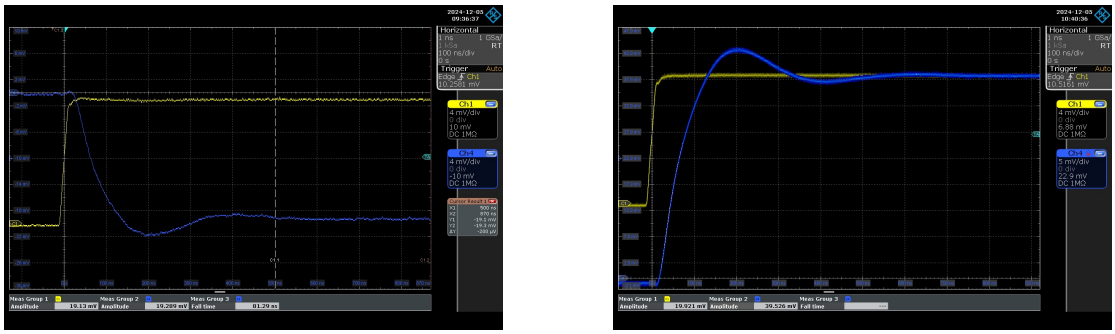


Figura 5.2: Onda quadra in entrata all'amplificatore OP27 con circuito invertente e guadagno 1 (sinistra) e circuito non invertente con guadagno 2 (destra).

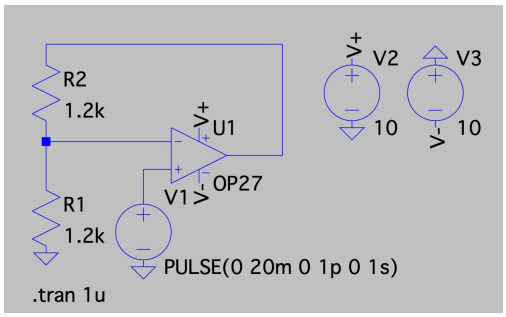


Figura 5.3: Circuito utilizzato nella simulazione. Si sono impostati i medesimi parametri presenti nella misurazione eseguita in laboratorio.

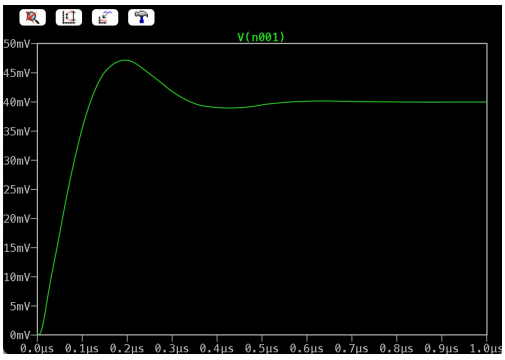


Figura 5.4: Comportamento ottenuto dalla simulazione. Si noti la corrispondenza con la figura 5.2.

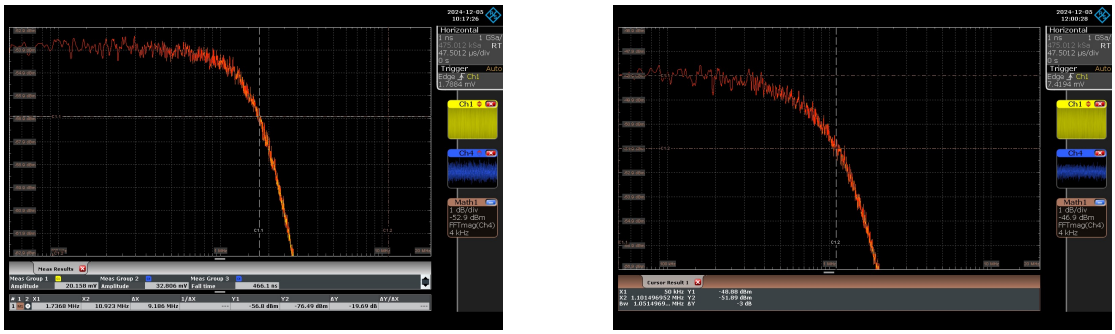


Figura 5.5: FFT dell'oscilloscopio con circuito invertente e guadagno 4.6 (sinistra) e circuito non invertente con guadagno 8 (destra).

5.2 AD848

Si riportano in egual modo a quanto fatto sopra per l'OP27 i dati e le considerazioni compiute sulla caratterizzazione dell'amplificatore AD848.[11]

Configurazione	**Guadagno**	**frequenza di taglio** (MHz)
Non invertente	4.25	41.176
	11.00	15.909
	28.50	6.140
	57.67	3.035

Tabella 5.3: Risultati teorici ricavati dalle misurazioni delle resistenze.

[11] I calcoli e tutti i dati acquisiti sono riportati sul foglio di calcolo.

ONDA QUADRA			
Guadagno	**Stima**	**Frequenza di taglio (MHz)**	**Stima**
4.218	99.25%	21.633	52.54%
10.800	98.18%	12.016	75.53%
28.161	98.81%	5.323	86.68%
56.536	98.04%	2.670	87.99%

FFT	
Frequenza taglio (MHz)	**Stima**
18.91	45.93%
11.72	73.66%
5.64	91.90%
2.67	87.98%

IMPULSO SINUSOIDALE			
Guadagno	**Stima**	**Frequenza taglio (MHz)**	**Stima**
4.218	99.25%	21.633	52.54%
10.800	98.18%	12.016	75.53%
28.161	98.81%	5.323	86.68%
56.536	98.04%	2.670	87.99%

Tabella 5.4: Stima delle frequenze di taglio con onda quadra (sopra), FFT (centro) e impuslo sinusoidale (sotto).

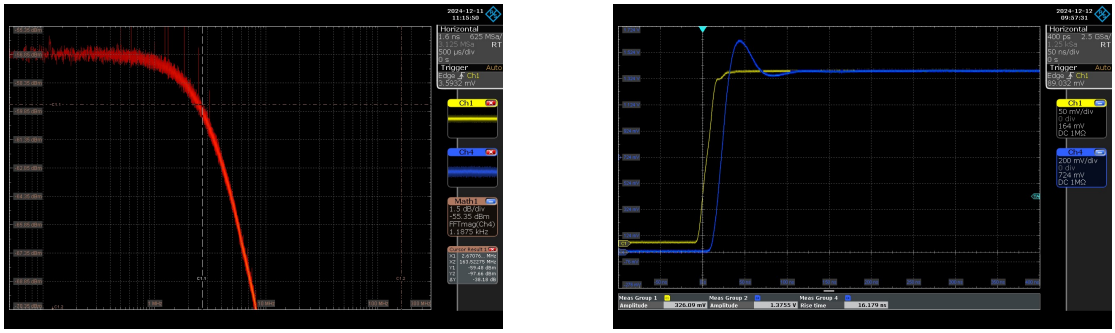


Figura 5.6: FFT dell'oscilloscopio con guadagno 57.7 (sinistra) e onda quadra in entrata all'amplificatore AD848 con guadagno 4.25 (destra).

Si nota anche in questo caso che l'accettabilità delle misurazioni compiute della frequenza di taglio indichi un discostamento dai valori predetti teoricamente. Le motivazioni sono uguali a quelle riportate nella sezione precedente per l'altro amplificatore. Si riportano in figura 5.6 alcune immagini a supporto della tesi per il campionamento effettuati con l'AD848.

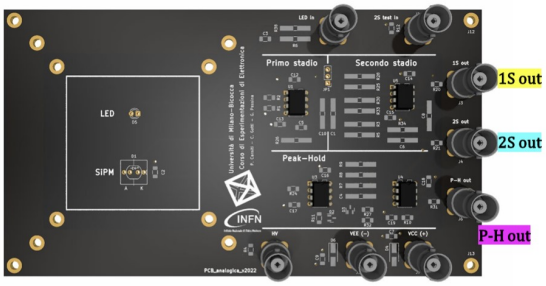


Figura 6.1: 1S: Primo stadio. 2S: Secondo stadio. P-H: peack and hold

6 Progetto finale

Il progetto finale consiste nel realizzare una catena di acquisizione [11, pp. 32–36] di singoli fotoni basata su tecnologia SiPM, una tipologia di rivelatore che genera un impulso proporzionale al numero di fotoni rilevati [11, pp. 27–31]. Il segnale viene amplificato con un circuito di condizionamento (figura 6.1) e allungato con un circuito di peak and hold [11, pp. 37–41]. Il segnale viene campionato e digitalizzato con l'ADC del microcontrollore e i dati vengono trasmessi tramite USART al computer. Nella sezione 4.5 viene mostrata una possibile implementazione software dell'acquisizione dati[12].

Nel nostro caso di test, la sorgente dei fotoni è un LED blu pilotato da un generatore di impulsi [11, pp. 20–25]. L'implementazione del progetto prevede l'esecuzione di diversi compiti intermedi: la calibrazione del guadagno degli amplificatori, dopo la previa installazione dell'apparato sperimentale, la caratterizzazione dei conteggi di buio del SiPM, la misurazione dei conteggi alimentando il LED tramite oscilloscopio, la scrittura del codice del microcontrollore per l'acquisizione di misure ripetute, con il confronto con i risultati ottenuti tramite oscilloscopio [11, pp. 42–44].

6.1 Conteggi di buio

Al fine di caratterizzare il sistema di acquisizione del progetto finale si procede con l'analisi dei conteggi di buio, ovverosia segnali rilevati come uno o pochi fotoni, ma dovuti all'agitazione termica. Si compie un'analisi riguardante la frequenza di conteggi di buio, calcolata su un numero di 1000 conteggi. Si riportano i valori misurati in tabella 6.1, mentre in tabella 6.2 vengono riportati la media e l'errore della frequenza e del rate dei conteggi.

Osservando il data sheet [12, p. 3], si può notare come il valore di dark count rate (DCR) sia fornito a 25 °C. In riferimento a *Introduction to the Silicon Photomultiplier (SiPM)* [13, p. 10], si trova l'andamento DCR (*Dark Count Rate*) in funzione della temperatura e si legge: *An increase in temperature will increase the dark count rate, and the converse is also true: for every* 10 °C *reduction in device temperature, there is a* 50 % *decrease in the dark count rate* (ulteriore conferma: [14, pp. 10, 16]). Stimata la temperatura dell'ambiente di lavoro pari a 20 °C, DCR risulta essere 66.667 kHz distante 1.47σ dal valore da noi calcolato.

Tempo acquisizioni (ms)	Rate (acq/ms)
19.23	0.01923
16.83	0.01683
15.75	0.01575

Tabella 6.1: Dati delle acquisizioni

Media rate	0.017
Deviazione standard	0.002
Frequenza (kHz)	57.886
Errore	5.975

Tabella 6.2: Statistiche delle acquisizioni

[12] I codici usati in laboratorio sono riportati sul repository GitHub.

Un'ulteriore analisi compiuta riguarda la dipendenza dell'ampiezza dei segnali di buio rispetto alla tensione di overvoltage. Si riportano in tabella 6.3 i valori dell'intercetta della retta e il valore ad essa associato[13].

q	σ_q	**accettabilità** t**-test**
-0.009	0.462	96.5%

Tabella 6.3: valore ed errore associato all'intercetta dell'interpolazione

Si può confrontare l'intercetta con il valore aspettato di zero. Il risultato ottenuto è accettabile quindi la tensione di breakdown è confrontabile con valore riportato del SiPM, pari a 51.2 V. La dipendenza lineare delle due grandezze è verificata dall'accettabilità del test del chi-quadro.

6.2 Misurazioni finali

L'obiettivo delle misurazioni finali dell'esperimento è la costruzione dello spettro delle ampiezze tramite istogramma [11, pp. 45, 46].

Si utilizzano due tecniche diverse sul lato codice per campionare le ampiezze dei segnali in uscita [11, p. 46]: in primo luogo con un sistema software di trigger e pretrigger implementato sull'ampiezza ricevuta dalle misurazioni condotte con il fotomoltiplicatore, successivamente invece si utilizza un sistema di trigger con detector analogico sull'impulso di stimolo di lampeggiamento del LED. Una volta campionate le funzioni d'onda si seleziona il massimo di una finestra di valori in corrispondenza dell'istante di trigger e con i valori ottenuti si costruisce un istogramma in Matlab.

Si utilizza inoltre la visualizzazione dei segnali anche da oscilloscopio per controllare la buona riuscita di campionamento dal lato microcontrollore. Si riporta in figura 6.2 un esempio di ciò che si visualizzava sull'oscilloscopio.

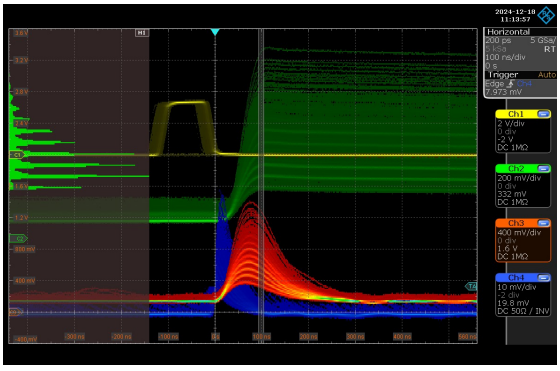


Figura 6.2: Il canale 1 mostra l'impulso generato dall'oscilloscopio in entrata al LED. Il canale 2 mostra l'ampiezza riportata tramite canale 3 a cui viene applicato il peak-and-hold. Il canale 3 mostra la tensione in uscita dal secondo stadio. Il canale 4 mostra la tensione in uscita dal primo stadio.

Si riporta in figura 6.4 quattro istogrammi generati dai campionamenti ottenuti con un sistema software con trigger digitale. Tutti gli istogrammi a venire hanno sull'asse delle ascisse i valori di tensione e sull'asse delle ordinate il conteggio dei singoli bin.

Sapendo che l'ampiezza generata dalle funzioni d'onda acquisite con il microcontrollore è proporzionale all'energia del fotone acquisito, si può notare come sia visibile la discretizzazione dello spettro. Nella figura 6.3, si può notare come il primo picco sia molto più alto rispetto agli altri: si suppone che questo sia dovuto alla modalità di acquisizione applicata. Vengono campionati tutti i segnali che superano la soglia impostata senza fare distinzione tra quelli di buio e quelli dovuti al lampeggiamento del LED. Si acquisiscono i segnali di buio e si sovrappone l'istogramma ottenuto con quello del campionamento precedente. Si può notare come i due istogrammi si sovrappongano in gran parte: questo ci porta a confermare l'ipotesi formulata.

[13] I dati misurati sono riportati sul foglio di calcolo, mentre l'analisi dati è disponibile nel foglio python.

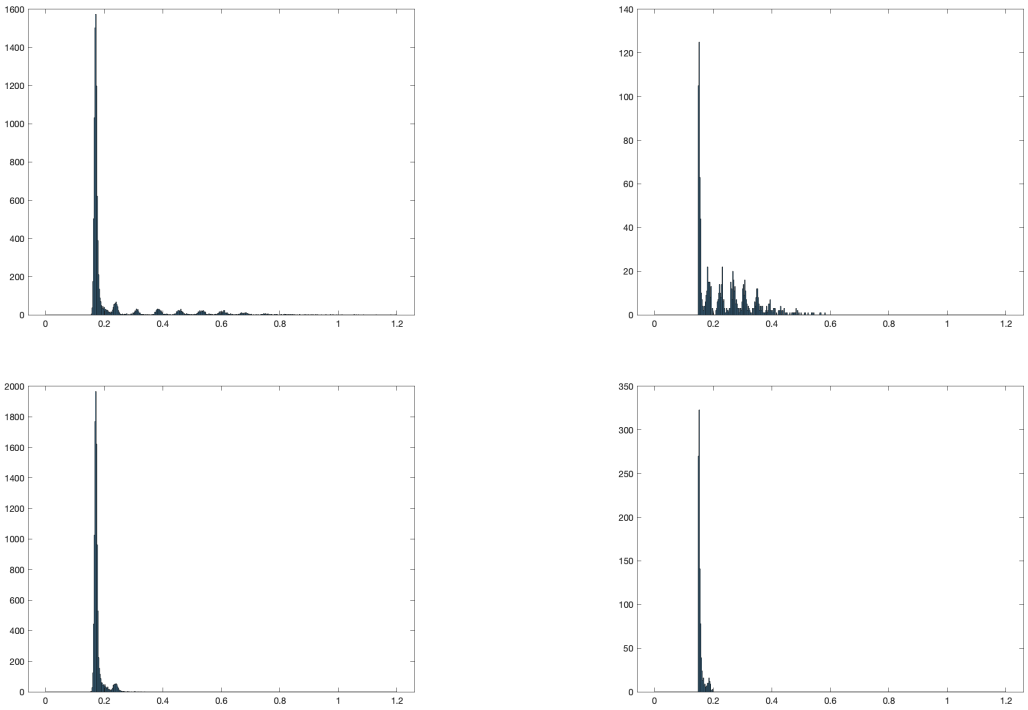


Figura 6.3: Campionamento con LED attivo (colonna sinistra) e spento (colonna destra) ad una frequenza di impulso di 8 kHz (riga superiore). Medesimi campionamenti con impulso a 10 kHz (riga inferiore). Si utilizza una tensione di alimentazione per SiPM di 5.2 V (1 OV) e un'ampiezza dell'impulso di 3.3 V.

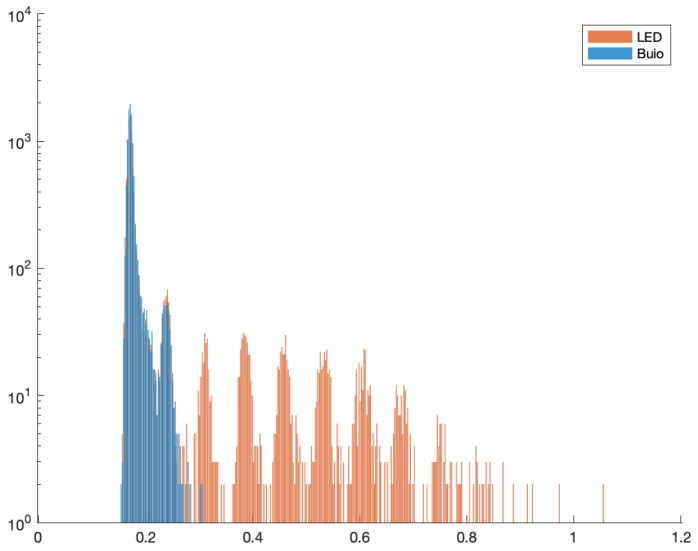


Figura 6.4: Sovrapposizione tra due spettri: quello costruito dai conteggi di buio (azzurro) e quello dato dal campionamento totale con trigger digitale (arancione). Si è scelta una scala logaritmica sulle ordinate per facilitare la lettura ed evidenziare il contrinuto dato dai conteggi di buio. L'istogramma corrisponde al solo campionamento effettuato con sorgente impulsata a 8 kHz.

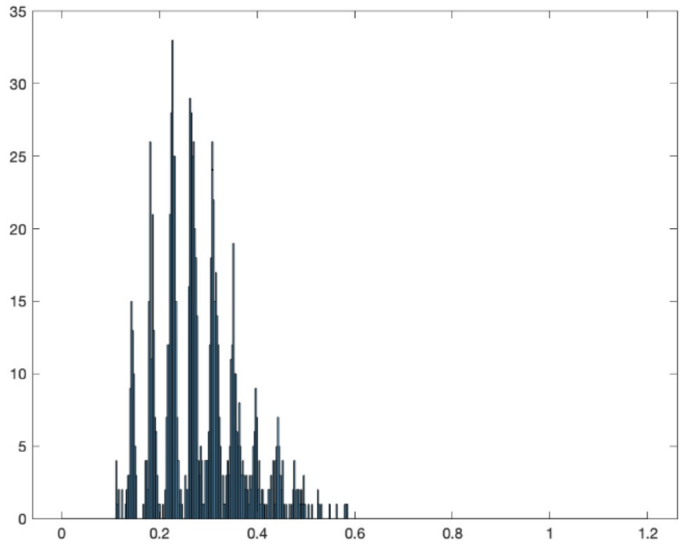


Figura 6.5: Spettro di ampiezze con acquisizione con trigger analogico su impulso a 8 kHz di ampiezza 3.3 V e alimentazione SiPM di 5.2 V (1 OV).

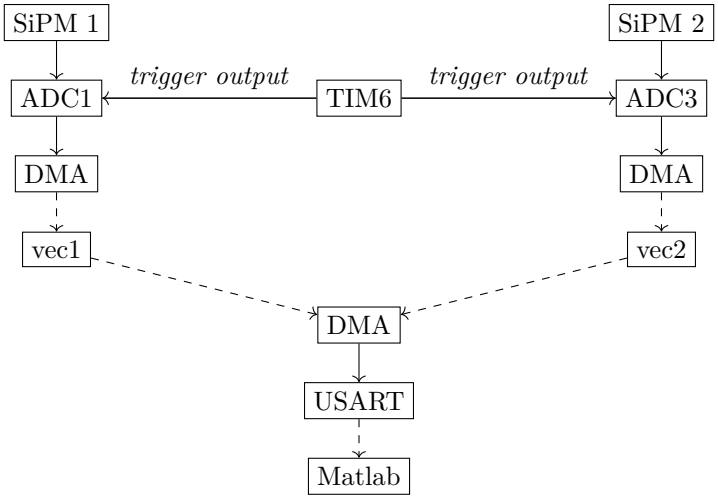
Si procede dunque con il secondo metodo di campionamento: utilizzando il trigger sull'impulso del LED si selezionano solamente gli eventi dovuti al lampeggiamento. Si riporta in figura 6.5 l'istogramma ottenuto.

Nella figura 6.5, si può notare che si ottiene uno spettro a pettine con picchi equidistanti tra loro e simile in forma a quello ottenuto con l'immagine da oscilloscopio (cfr. figura 6.2). Il passo fisso tra picchi è indice del fatto che si stiano acquisendo dei fotoni con energia quantizzata. Sapendo che l'ampiezza di acquisizione è proporzionale all'energia dei fotoni si può dedurre che esiste una corrispondenza biunivoca tra il numero del picco (immaginando di numerarli da sinistra a destra nell'immagine) e il numero di fotoni campionati.

7 Approfondimenti

7.1 Coincidenze

In questa esperienza si vuole campionare contemporaneamente due segnali diversi generati da una stessa sorgente. Per perseguire questo obiettivo si è aggiornato l'apparato sperimentale precedentemente descritto aggiungendo un SiPM per la ricezione dei fotoni provenienti dal LED e togliendo il secondo stadio a entrambi. Si utilizza la tecnica delle coincidenze per la gestione del trigger. Per la gestione software dell'acquisizione si rimanda al seguente schema.



La principale problematica riscontrata lato codice è l'essere certi che l'impulso campionato sia lo stesso per entrambi i fotomoltiplicatori; si imposta dunque un delay minimo tra il superamento della soglia di trigger di un SiPM e dell'altro per verificare che la coincidenza sia avvenuta. La ricostruzione dei dati è analoga a quella nelle sezioni sopra riportate. Si mostrano gli istogrammi ricavati in figura 7.1.

Si può notare che i due istogrammi non siano perfettamente sovrapponibili: ciò potrebbe essere dovuto al fatto che il led non fosse perfettamente centrato tra i due SiPM e che la tensione di breakdown fosse diversa per i due fotomoltiplicatori.

7.2 Cristalli

Nell'apparato sperimentale si monta al posto del led un cristallo scintillatore direttamente connesso con i due fotomoltiplicatori. Viene poi posta una sorgente radioattiva al ^{22}Na vicino all'apparato: il cristallo scintillatore emette fotoni al passaggi di particelle cariche, che vengono emesse dalla sorgente. L'obiettivo è quello di rilevare i fotoni emessi, usando la tecnica delle coincidenze.

Il Sodio-22 emette positroni β^+ che si annichilano con elettroni liberi e^-, producendo due raggi gamma da 511 keV, che si possono ricombinare in un gamma da 1276 keV (figura 7.2). Quest'ultimo raggio gamma è meno rilevabile per la poca probabilità che venga prodotto ed essere successivamente rilevato in coincidenza.

Si utilizza inizialmente un cristallo YAG:Ce, *granato di ittrio e alluminio drogato con cesio*. Si riporta in figura 7.3 un esempio di acquisizione con visualizzazione da oscilloscopio compiuta tramite la tecnica di coincidenza. Si riportano in figura 7.4 gli istogrammi ottenuti nei campionamenti.

Si utilizza successivamente un cristallo BGO, *bismuto germanato*, con dimensioni caratteristiche maggiori del cristallo utilizzato precedentemente; si cambiano nella configurazione sperimentale anche i fotomoltiplicatori, sostituendo il modello S13360-1350CS (1.3 mm × 1.3 mm) con S13360-6050CS (6 mm × 6 mm). Date le dimensioni del nuovo cristallo, il rumore di fondo dato dai conteggi di buio risulta essere molto maggiore rispetto al modello utilizzato in precedenza.

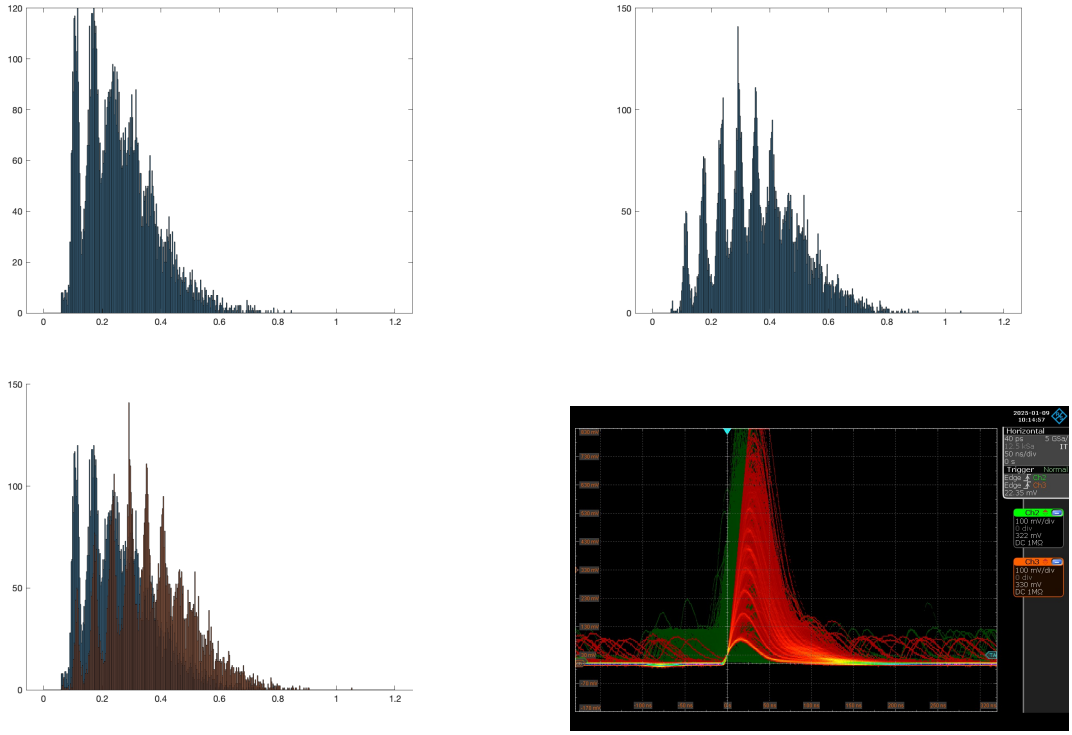


Figura 7.1: Riga superiore: campionamento di ADC1 (sinistra) e ADC3 (destra). Riga inferiore: sovrapposizione dei due spettri (sinistra) e campionamento da oscilloscopio con persistenza (destra). Sorgente: LED con impulso a 50 kHz di ampiezza 2.7 V e alimentazione SiPM di 5.4 V (2 OV).

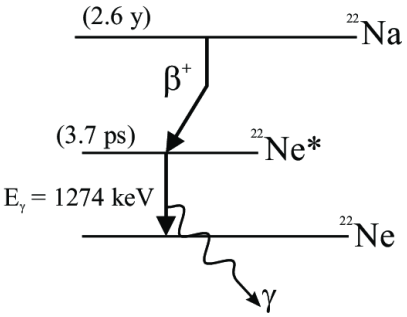


Figura 7.2: Diagramma decadimento di ^{22}Na

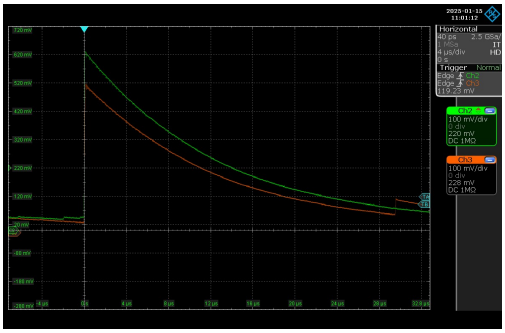


Figura 7.3: Esempio di campionamento da oscilloscopio con cristallo YAG

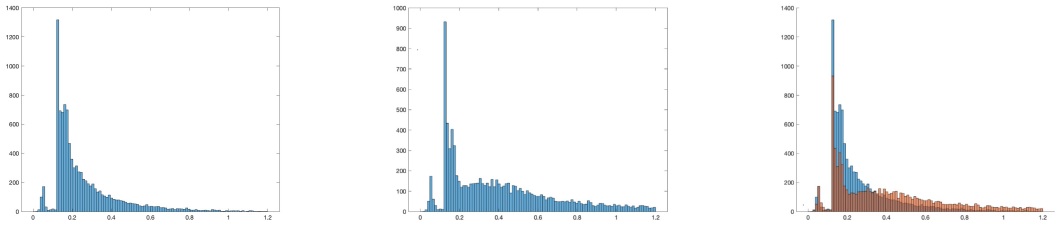


Figura 7.4: Da sinistra: spettro da SiPM1 (ADC1); spettro da SiPM2 (ADC3); sovrapposizione tra gli spettri (arancione-SiPM2, azzurro-SiPM1). Alimentazione SiPM: 5.4 V (2 OV).

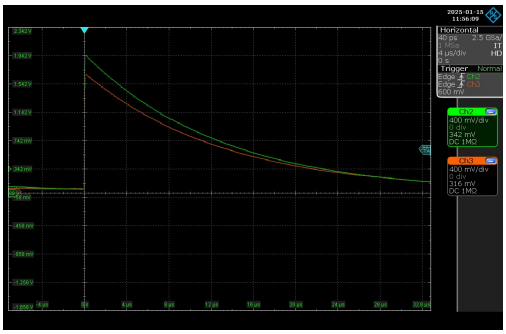


Figura 7.5: Esempio di campionamento da oscilloscopio con cristallo BGO

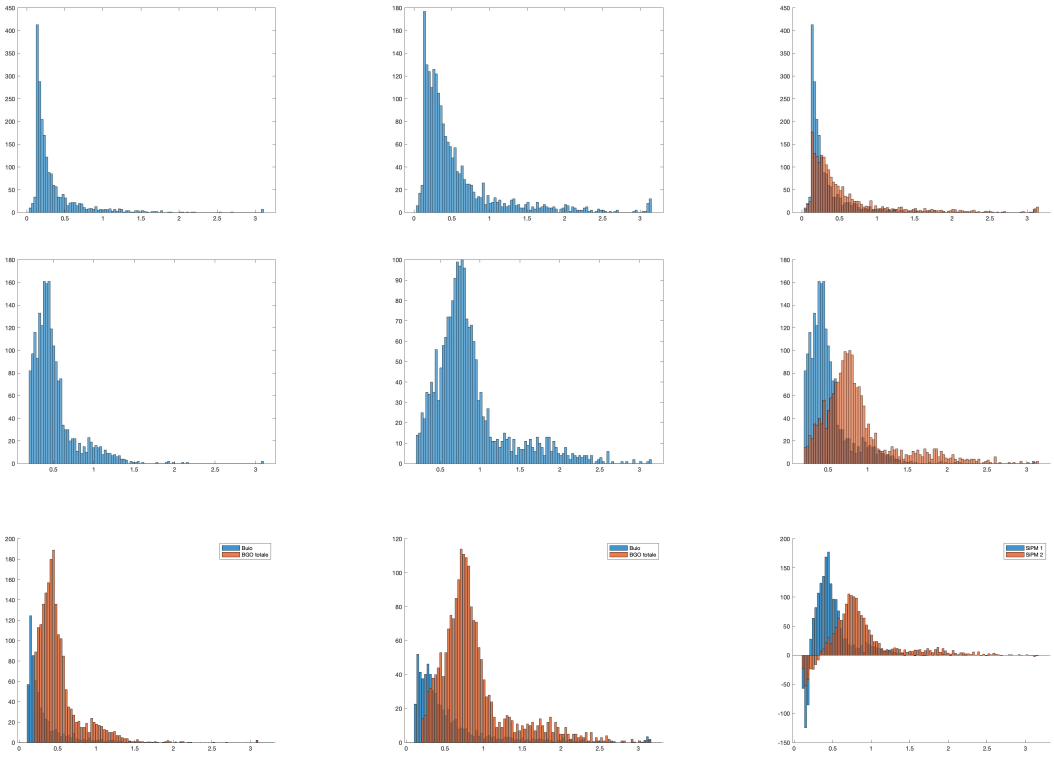


Figura 7.6: Colonna di sinistra: campionamenti effettuati da ADC1. Colonna centrale: campionamenti effettuati da ADC3. Colonna destra: sovrapposizione di entrambe con ADC1 in azzurro e ADC3 in arancione. Riga superiore: acquisizioni di buio in 106.18 s. Riga centrale: acquisizioni in conicidenza in 33.45 s. Riga inferiore: sovrapposione tra buio e dati acquisiti (sinistra e centro); sottrazione tra acquisizioni di buio e confronto tra ADC (destra). Alimentazione SiPM: 5.4 V (2 OV).

Si sono svolti due campionamenti diversi: in assenza e in presenza della sorgente ^{22}Na. Fissato il numero di misure, si è registrato il tempo di acquisizione; in questo modo, normalizzati entrambi gli istogrammi fissando un unico sample-time, è stato possibile confrontare le acquisizioni. Si riporta in figura 7.5 un esempio di acquisizione con visualizzazione da oscilloscopio compiuta tramite la tecnica di coincidenza.

Gli istogrammi ottenuti nei campionamenti sono riportati in figura 7.6. Sottraendo i conteggi di buio dalle acquisizioni con sorgenti (figura 7.6 in basso a destra), si può notare come ci siano dei bin con "conteggi negativi" (valori negativi nell'istogramma). Questa problematica è probabilmente dovuta al fatto che le soglie di trigger utilizzate per i due campionamenti sono differenti: maggiore per i conteggi con sorgente, minore per i conteggi di buio.

Riferimenti bibliografici

[1] G. Pessina C. Gotti. *I microcontrollori (parte B)*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/I_microcontrollori_Parte_B_2425.pdf.

[2] STMicroelectronics. *UM2407 User Manual: STM32H7 Nucleo-144 boards (MB1364)*. Ver. Rev 4. Ott. 2023. URL: https://www.st.com/resource/en/user_manual/um2407-stm32h7-nucleo144-boards-mb1364-stmicroelectronics.pdf.

[3] G. Pessina C. Gotti. *I microcontrollori (parte A)*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/I_microcontrollori_Parte_A_2425.pdf.

[4] G. Pessina C. Gotti. *Configuratore*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/Note_sul_configuratore_2425.pdf.

[5] STMicroelectronics. *NUCLEO 144 H7 LEGACY Schematic: MB1364-H743ZI*. Ver. Rev A4. Ott. 2019. URL: https://www.st.com/resource/en/schematic_pack/mb1364-h743zi-e01_schematic.pdf.

[6] STMicroelectronics. *RM0433 Reference Manual: STM32H742, STM32H743/753 and STM32H750 Value line advanced Arm-based 32-bit MCUs*. Ver. Rev 8. Gen. 2023. URL: https://www.st.com/resource/en/reference_manual/rm0433-stm32h742-stm32h743753-and-stm32h750-value-line-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.

[7] G. Pessina C. Gotti. *I convertitori*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/I_convertitori_2425.pdf.

[8] G. Pessina C. Gotti. *Acquisizioni con ADC*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/Acqisizione_con_ADC_H743ZI2_2425.pdf.

[9] STMicroelectronics. *STM32H742xI/G STM32H743xI/G Datasheet: 32-bit Arm Cortex-M7 480MHz MCUs, up to 2MB flash, up to 1MB RAM, 46 com. and analog interfaces*. Ver. Rev 10. Mar. 2023. URL: https://www.st.com/resource/en/datasheet/dm00387108.pdf.

[10] G. Pessina C. Gotti. *Amplificatori Reazionati: Analisi Tempo-Frequenza*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/Amplificatori_reazionati_analisi_tempo_frequenza_B_2425.pdf.

[11] G. Pessina C. Gotti. *Dispositivi a Stato Solido in Modo Euristico*. 2025. URL: https://pessina.mib.infn.it/Corsi_del_III_anno/CorsoStrumentazioneElettronica/Corso_2425/Dispositivi_a_stato_solido_in_modo_euristico_2425.pdf.

[12] Hamamatsu Photonics. *S13360 Series Photodiodes - Data Sheet*. Ver. Rev A. Nov. 2024. URL: https://www.hamamatsu.com/content/dam/hamamatsu-photonics/sites/documents/99_SALES_LIBRARY/ssd/s13360_series_kapd1052e.pdf.

[13] onsemi. *AN-9770: High-Side Current Sensing with the NCS2120*. Ver. Rev 9. Ago. 2023. URL: https://www.onsemi.com/pub/collateral/and9770-d.pdf.

[14] M. Calvi et al. «Single photon detection with SiPMs irradiated up to 10^{14} cm^{-2} 1-MeV-equivalent neutron fluence». In: *arXiv preprint arXiv:1805.07154* (2019). URL: https://arxiv.org/abs/1805.07154.